

❖排序是很常见的一类问题（并不局限于排序本身）

[例7.1] 有1亿个随机给出的浮点数，请找出其中最大的1万个。

方法1： 简单选择法（1万亿）

方法2： 总比较次数为 $N-1+(N-2)+\dots+(N-10000)$ 次。当N为1亿时，大约为1万亿次。

➤ 比如，以1百万为一个块，分为100块，分别对这100块数据进行排序。由于只需要得到最大的1万个数，故每块排完后可以只要前1万个数，再从这100块中取出最大的1万个就可以了。N=1亿， $\log_{10} 10000 = 4$ ，很快被排除！比如只有1/10需要替换。

➤ 当N为100万时， $O(N \log N)$ 是2000万，所以求解101块百万数据的排序问题，时间大约是20亿次运算。

过滤所有数据需要14亿？

方法3： 堆选择（2.4亿）

➤ 先读出1百万个数，建立初始堆（100万运算量），对剩下的近1亿个数进行过滤：每次读入剩下的一个数，如果该数小于等于这1万个数的最小值，则继续读下一个数；否则，用该数替代1万个数里的最小值。

➤ 100万（1百万建堆）+ 1亿（顺序过滤）+ 14 * 1千万，共约2.4亿。

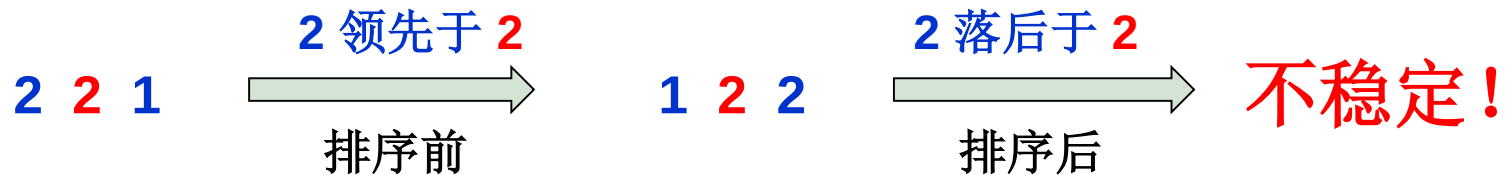
❖ 简单选择排序

➤ 时间复杂性 $T(n) = O(n^2)$

➤ 空间复杂性 $S(n) = O(1)$

➤ 稳定性：不稳定。反例如下：

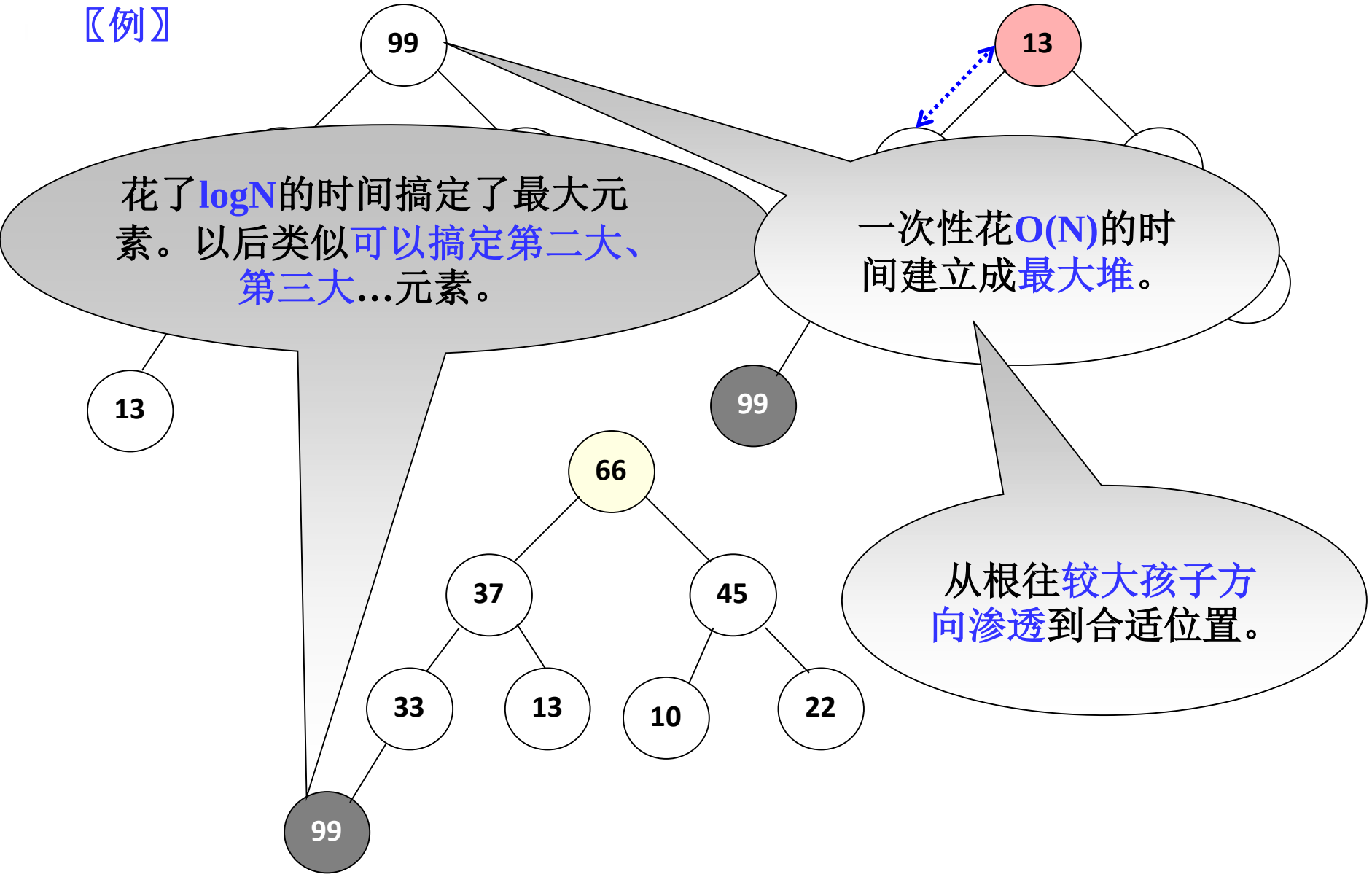
对初始已经有序的序列
没有实质性的省时间优势。



❖ 堆排序（属于选择排序大类）

➤ 堆排序的核心思想是：利用最大堆（或者最小堆）输出堆顶元素，即最大值（或最小值），将剩余元素重新生成最大堆（或者最小堆），继续输出堆顶元素，重复此过程，直到全部元素都已输出，得到的输出元素序列即为有序序列。

【例】



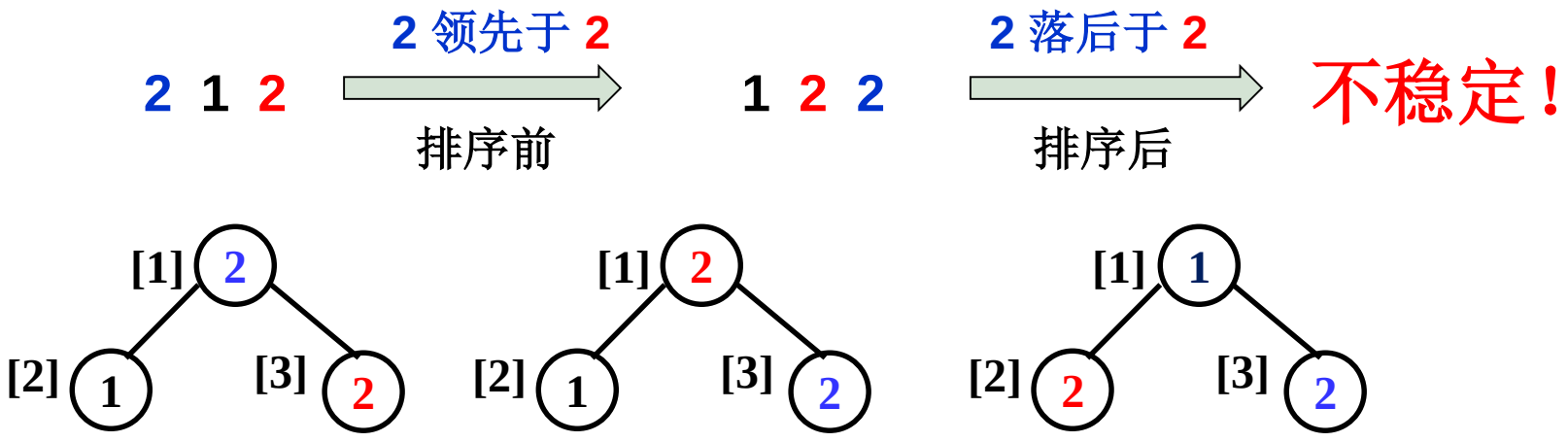
```
void Adjust( ElementType A[], int i, int N )
{
    /* 对A[]中的前N个元素从第i个元素开始向下迁移调整 */
    int Child;
    ElementType temp;
    for( temp = A[i]; (2*i + 1) < N; i = child){
        Child = (2*i + 1); /* 左孩子节点 */
        if ( (Child != N-1) && A[Child+1] > A[Child])
            Child++;
        if( temp < A[Child])
            A[i] = A[Child];
        else break;
    }
    A[i] = temp; /*将temp放到当前位置 */
}

void HeapSort( ElementType A[], int N ) /* 堆排序 */
{
    int i;
    ElementType temp;
    for( i = (N-1)/2; i >= 0; i-- ) /*从有儿子的最后一个结点开始*/
        Adjust(A, i, N); /* 建立最大堆*/
    for( i = N-1; i > 0; i-- ){
        /* 将堆顶元素A[0]与当前堆的最后一个元素A[i]换位 */
        temp = A[0]; A[0] = A[i]; A[i] = temp;
        /* 将有i个元素的新堆从根结点向下过滤调整 */
        Adjust(A, 0, i);
    }
}
```

与DeleteMax(MaxHeap H) 类似。

❖ 堆排序（属于选择排序大类）

- 时间复杂性 $T(n) = O(n \log n)$
- 空间复杂性 $S(n) = O(1)$
- 稳定性：不稳定。反例如下：



❖ 简单插入排序

```
void InsertionSort ( ElementType A[ ], int N )
{   /* 插入排序 */
    int i, j;
    ElementType temp;
    for ( i = 1; i < N; i++ ) {
        temp = A[i];          /* 取出未排序序列中的第一个元素*/
        for ( j = i; (j > 0) && (temp < A[j-1]); j-- )
            A[j] = A[j-1]; /*依次与已排序序列中元素比较并右移*/
        A[j] = temp; /* 放进合适的位置 */
    }
}
```

- 最坏情形： 输入的A[] 是逆序. $T(N) = O(N^2)$
- 最好情形： 输入的A[] 已经有序. $T(N) = O(N)$
- 空间复杂性 $S(n) = O(1)$
- 稳定性： 稳定。

❖ 简单插入排序

【Definition】 An **inversion** in an array of numbers is any ordered pair (i, j) having the property that $i < j$ but $A[i] > A[j]$.

【Example】 Input list 34, 8, 64, 51, 32, 21 has **9** inversions.
(34, 8) (34, 32) (34, 21) (64, 51) (64, 32) (64, 21) (51, 32) (51, 21) (32, 21)

There are **9** swaps needed to sort this list by insertion sort.

Swapping two adjacent elements that are out of place removes **exactly one** inversion.

$T(N, I) = O(I + N)$ where I is the number of inversions in the original array.

Fast if the list is **almost sorted**.

❖ 简单插入排序

【Theorem】 The average number of inversions in an array of N distinct numbers is $N(N-1)/4$.

【Theorem】 Any algorithm that sorts by exchanging adjacent elements requires $\Omega(N^2)$ time on average.

❖ 希尔排序——by Donald Shell

【例】：

81	94	11	96	12	35	17	95	28	58	41	75	15
----	----	----	----	----	----	----	----	----	----	----	----	----

5-sort

35	17	11	28	12	41	75	15	96	58	81	94	95
----	----	----	----	----	----	----	----	----	----	----	----	----

3-sort

28	12	11	35	15	41	58	17	94	75	81	96	95
----	----	----	----	----	----	----	----	----	----	----	----	----

1-sort

11	12	15	17	28	35	41	58	75	81	94	95	96
----	----	----	----	----	----	----	----	----	----	----	----	----

✎ 定义一个递增序列： $h_1 < h_2 < \dots < h_t$ ($h_1 = 1$)

✎ 分步骤进行“间隔 h_k -插入排序”， $k = t, t-1, \dots, 1$

注意：“间隔 h_k -插入排序”的结果留给“间隔 h_{k-1} -插入排序”

👉 Shell's increment sequence:

$$h_t = \lfloor N / 2 \rfloor, h_k = \lfloor h_{k+1} / 2 \rfloor$$

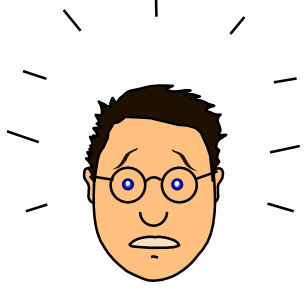
```
void ShellSort( ElementType A[ ], int N, int Incre[ ], int M )
{ /* 希尔排序*/
    /* 其中Incre[]为包含M个增量的序列, 递减有序, 且最后一个元素为1 */
    int i, j, k, Increment;
    ElementType temp;
    for ( i = 0; i < M; i++ ) {
        Increment = Incre[ i ];          /* 选择该趟排序需要的增量 */
        for ( j = Increment; j < N; j++ ) {
            temp = A[ j ];
            for ( k = j; k - Increment >= 0; k -= Increment ) {
                if ( temp >= A[ k - Increment ] ) break;
                else
                    A[ k ] = A[ k - Increment ];
            }
            A[k] = temp;
        } /* 结束一趟插入排序 */
    }
}
```

👉 Worst-Case Analysis:

【Theorem】 The worst-case running time of Shellsort, using Shell’s increments, is $\Theta (N^2)$.

【Example】 A bad case:

	1	9	2	10	3	11	4	12	5	13	6	14	7	15	8	16
8-sort	1	9	2	10	3	11	4	12	5	13	6	14	7	15	8	16
4-sort	1	9	2	10	3	11	4	12	5	13	6	14	7	15	8	16
2-sort	1	9	2	10	3	11	4	12	5	13	6	14	7	15	8	16
1-sort	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16

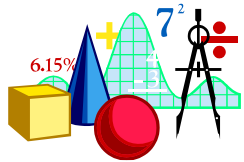


Pairs of increments are not necessarily relatively **prime**.
Thus the smaller increment can have little effect.

☞ Hibbard's Increment Sequence:

$h_k = 2^k - 1$ ---- consecutive increments have no common factors.

【Theorem】 The worst-case running time of Shellsort, using Hibbard's increments, is $\Theta (N^{3/2})$.



Conjectures:

☞ $T_{\text{avg-Hibbard}} (N) = O (N^{5/4})$

Shellsort is a very simple algorithm, yet with an extremely complex analysis. It is good for sorting up to moderately large input (tens of thousands).

☞ Sedgewick's best sequence is $\{1, 5, 19, 41, 109, \dots\}$ in which the terms are either of the form $9 \times 4^i - 9 \times 2^i + 1$ or $4^i - 3 \times 2^i + 1$. $T_{\text{avg}} (N) = O (N^{7/6})$ and $T_{\text{worst}} (N) = O (N^{4/3})$.

❖ 希尔排序

➤ 希尔排序算法的整体时间复杂度和增量序列的选取有关，目前并没有统一的最优增量序列。

➤ 当使用增量序列 $\{\lfloor N/2 \rfloor, \lfloor N/2^2 \rfloor, \dots, 1\}$ 进行希尔排序时，有例子表明，最差情况下的时间复杂度 $T_{\text{worst}}(n) = O(N^2)$ ；

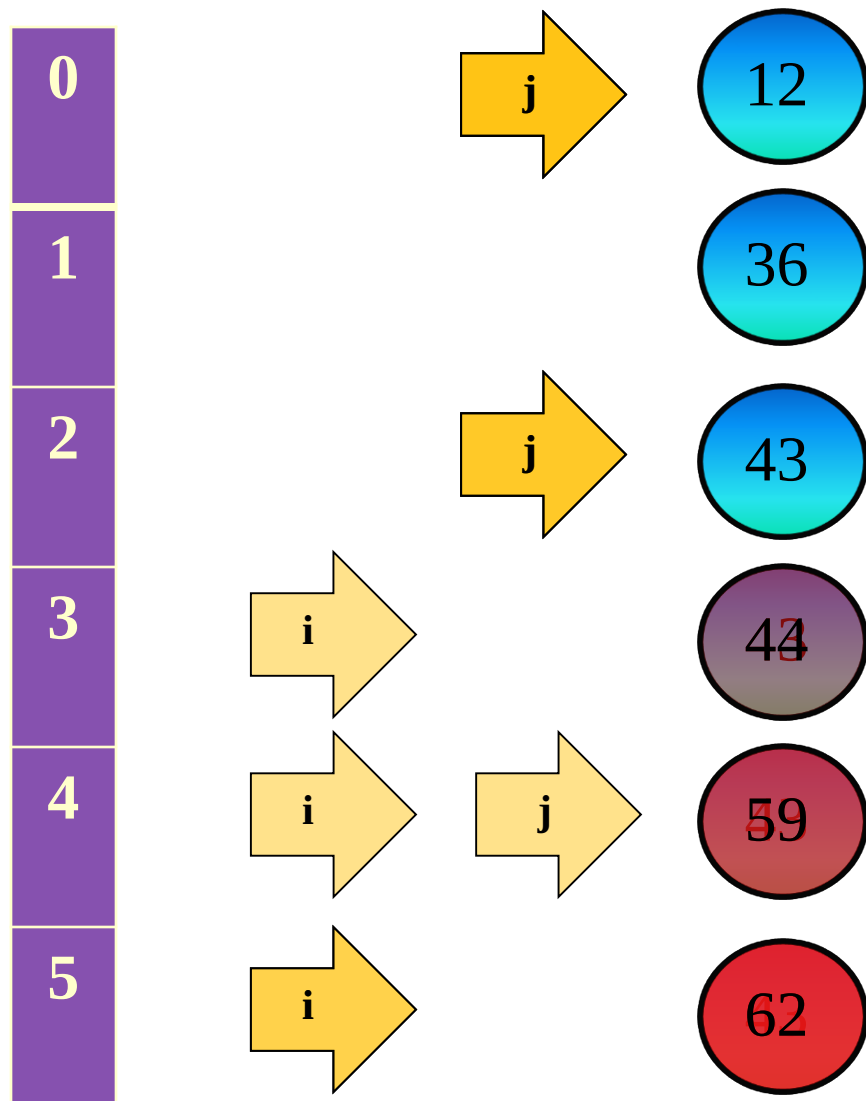
➤ 而当使用增量序列 $\{2^k-1, \dots, 7, 3, 1\}$ 时，最差情况下时间复杂度为 $T_{\text{worst}}(n) = O(N^{3/2})$ ，平均时间复杂度为 $T_{\text{average}}(n) = O(N^{5/4})$ 。

➤ 空间复杂性 $S(n) = O(1)$

➤ 稳定性：不稳定。

对初始基本有序的序列
可以提高时间性能。

❖ 冒泡排序



- 无交换发生，可以提前结束？
- 时间复杂性 $T(n) = O(n^2)$
- 空间复杂性 $S(n) = O(1)$
- 稳定性：稳定。

```
void BubbleSort ( ElementType A[ ], int N ) /*冒泡排序*/
```

```
{ int i, j ;
```

```
  bool flag;
```

```
  ElementType temp;
```

```
  for ( i = N - 1; i >= 0; i-- ) {
```

```
    flag = 0 ;
```

```
    /*标记该次循环中是否发生交换，若无，则说明整个序列有序*/
```

```
    for ( j = 0; j < i; j++ ) {
```

```
      /*每次循环找出一个最大元素，被交换到最下端*/
```

```
      if ( A[j] > A[j + 1] ) {
```

```
        temp = A[j] ; A[j] = A[j + 1] ; A[j + 1] = temp ;
```

```
        flag = 1 ;      /*若发生交换，则标记*/
```

```
      }
```

```
    }
```

```
    if ( !flag ) break ;    /*若没有发生交换，则跳出循环*/
```

```
  }
```

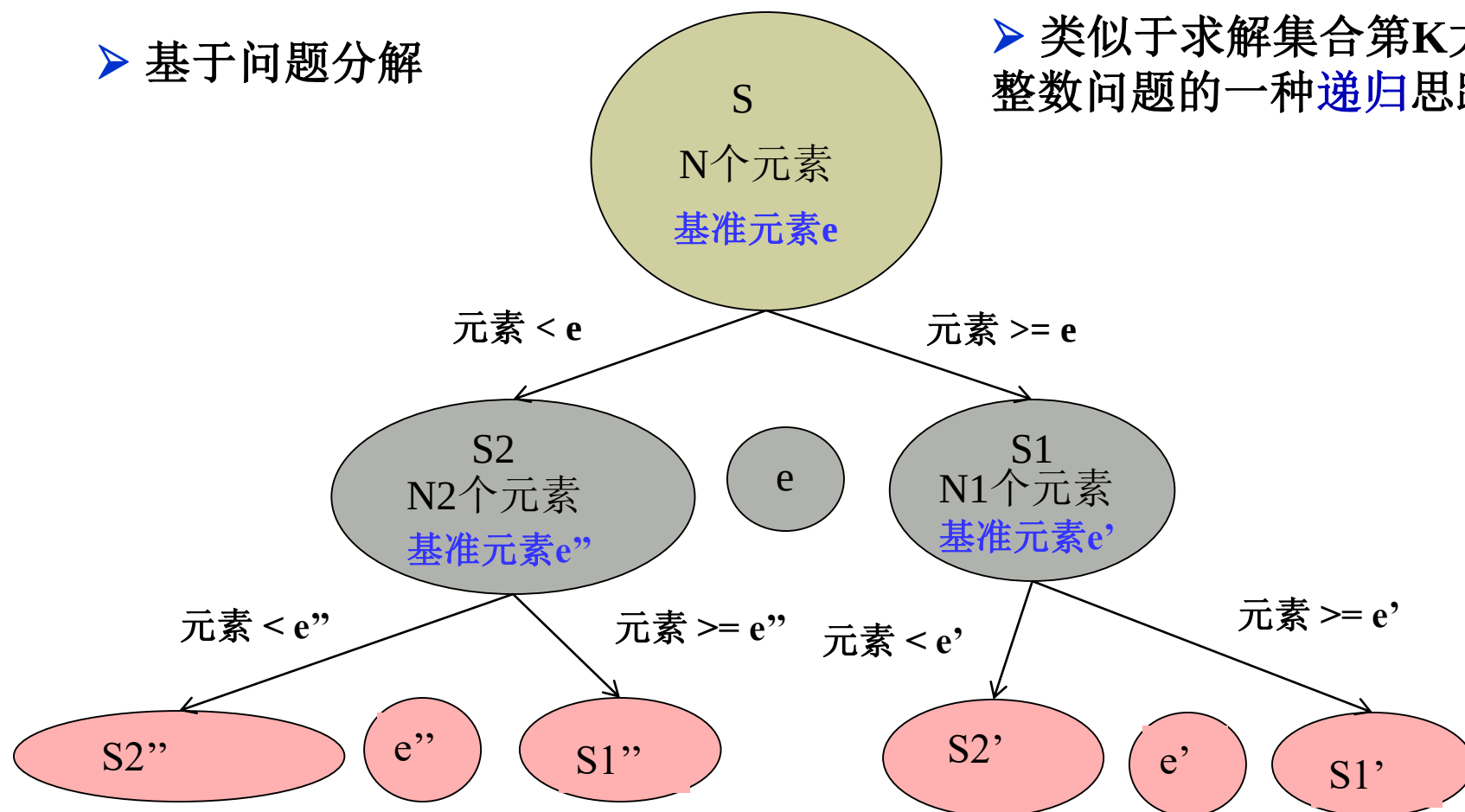
```
}
```

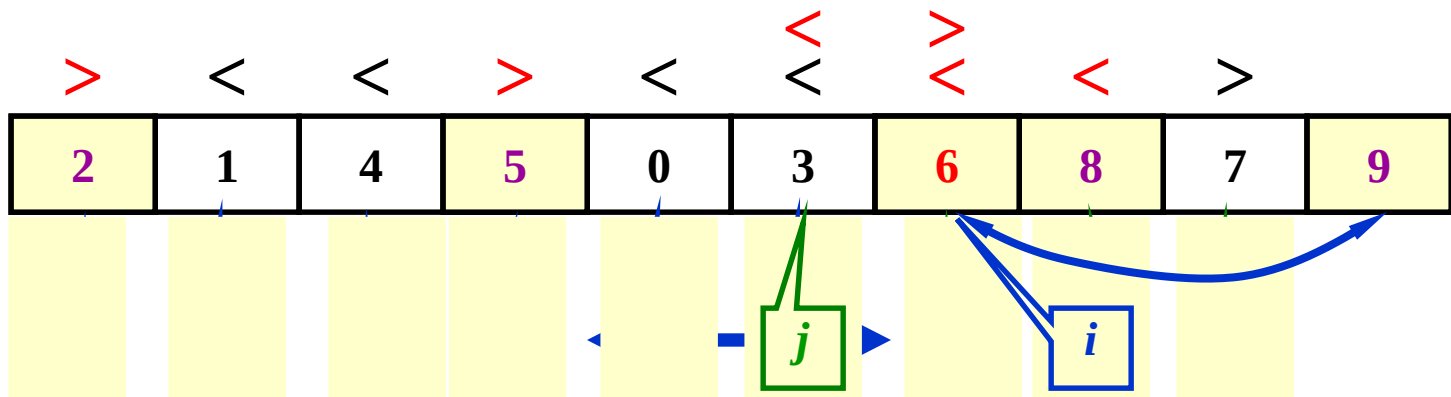
若不考虑提前结束，
可以忽略flag。

❖ 快速排序 (Quick Sort)

➤ 基于问题分解

➤ 类似于求解集合第K大整数问题的一种递归思路





❖ 分析时间复杂性

$$T(N) = T(i) + T(N - i - 1) + c N$$

👉 最坏情形:

$$T(N) = T(N - 1) + c N$$

➡ $T_{worst}(N) = O(N^2)$

👉 最好情形: [...] • [...]

为了避免最坏情形，确定基准时用“三者取中”的方法，即将 $A[low]$ 、 $A[high]$ 、 $A[(low + high) / 2]$ 三者关键字的中值

❖ 分析时间复杂性

$$T(N) = T(i) + T(N - i - 1) + cN$$

👉 平均情形:

对任意 i , 假定平均时间 $T(i)$ 是: $\frac{1}{N} \left[\sum_{j=0}^{N-1} T(j) \right]$

$$T(N) = \frac{2}{N} \left[\sum_{j=0}^{N-1} T(j) \right] + cN \quad \rightarrow \quad T(N) = O(N \log N)$$

❖ 分析空间复杂性

👉 由于快速排序需要进行至少 $\log_2 N$ 层的递归, 因此需要至少 $O(\log_2 N)$ 深度的栈空间。

❖ 稳定性: 不稳定

【例】给定 N 个元素的列表以及正整数 k . 求第 k 个大元素 (或中位数)。

```

void QSort ( ElementType A[ ], int Low, int High )
{
    /* 对A[Low]~A[High]进行快速排序 */
    ElementType Pivot = A[Low]; /* 简单取首元素为基准 */
    int Left = Low, Right = High;
    if ( Low >= High ) return;
    Swap ( &A[Low], &A[Right] ); /* 将基准与最后一个元素交换 */
    while (1){ /* 将序列中比基准小的移到基准左边，大的移到右边 */
        while ( (Low < High) && (Pivot >= A[Low]) ) Low++;
        while ( (Low < High) && (Pivot <= A[High]) ) High--;
        if ( Low < High ) Swap ( &A[Low], &A[High] );
        else break;
    }
    Swap ( &A[Low], &A[Right] ); /* 将最后的基准换到正确的位置 */
    /* 分别对两个子集进行快排 */
    QSort ( A, Left, Low - 1 );
    QSort ( A, Low + 1, Right );
}

void QuickSort( ElementType A[ ], int N )
{
    /* 快速排序 */
    QSort( A, 0, N-1 );
}
    
```

Swap是交换两个元素的函数。